Section A — Architecture

Abstract

This document describes a multitenant “answering over external data” system that combines Retrieval-Augmented Generation (RAG) with controlled tool usage to provide accurate and context-aware responses. The system integrates tenant-specific data with external APIs to support both static knowledge queries and real-time information needs.

It uses a modular architecture with an ingestion pipeline, hybrid retrieval (keyword + vector search), and a bounded agent layer that decides when to use retrieval, tools, or both. Documents are processed using structure-aware chunking and enriched with metadata to improve retrieval quality.

The design enforces strong multitenancy through tenant-level filtering, access control, and secure secret management, while maintaining performance through caching and cost controls. Overall, it balances simplicity, scalability, and reliability for production use.

1) Data ingestion and indexing

The ingestion pipeline is designed to be asynchronous and tenant aware. Data is collected from uploads or connectors, stored in raw format, and then processed through parsing and cleaning. The content is split into meaningful chunks using structure-aware chunking, which keeps headings and paragraphs intact instead of breaking them randomly.

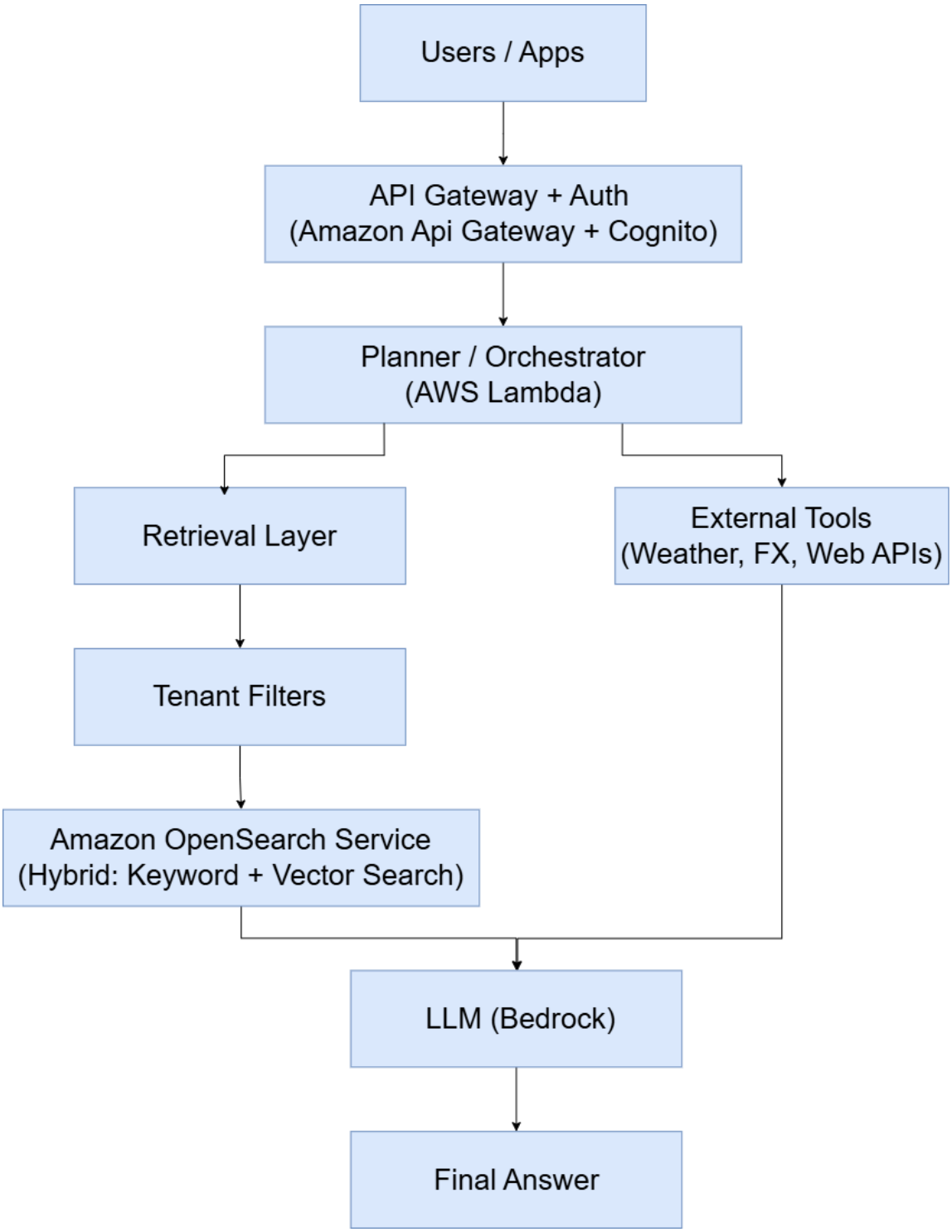
Each chunk is converted into embeddings and stored in a search index. The system uses hybrid search, which combines keyword search for exact matches and vector search for semantic similarity. A reranking step improves the results. Metadata such as tenant ID, document ID, and timestamps are stored with each chunk to support filtering and security.

Data Sources → Raw Storage → Parsing & Cleaning → Structure-aware Chunking → Embeddings → Search Index (with metadata)

2) Agentic layer

The agent layer is controlled and not fully autonomous. A planner decides whether to use RAG, tools, or both. For simple queries, rule-based logic is used for fast decisions. For more complex queries, a language model helps decide the best approach. Tools are designed with structured inputs and outputs, and limits are applied to avoid excessive usage. If a tool fails or times out, the system can retry or fall back to a simpler approach. This keeps the system reliable and avoids unnecessary delays.

Architecture



3) Cost, latency, and caching

To manage cost and latency, the system uses multiple caching layers. Prompt caching stores repeated instructions, vector caching stores search results and embeddings, and HTTP caching stores responses from external APIs.

Limits are applied on tokens, number of retrieved chunks, and tool usage. Most queries follow a simple flow of retrieve, rerank, and generate, which keeps response time low (around 1–2 seconds). More complex queries involving tools may take slightly longer.

4) Security and multitenancy

Security is enforced at every stage of the system. Each piece of data is tagged with a tenant ID, and all queries are filtered to ensure data isolation. Role-Based Access Control (RBAC) is used to manage permissions. For sensitive tenants, separate indexes can be used. Secrets are managed using secure services like Key Vault, and no credentials are stored in code. Audit logs track user actions, accessed data, and system responses for monitoring and compliance.

5) Observability

The system uses monitoring tools to track performance and reliability. Metrics such as latency, token usage, and cost are recorded. Failures like tool errors or missing results are also tracked. Evaluation is done using a small test dataset to check answer quality and reduce hallucinations. This helps maintain consistent performance over time.

6) Deployment

The system is containerized and deployed using a CI/CD pipeline. New versions are released using blue/green or canary deployment strategies. A small portion of traffic is first sent to the new version, and based on performance, it is either promoted or rolled back.

Final Summary

The system combines hybrid search, a controlled agent layer, and external tools to provide accurate answers. It ensures strong data isolation, efficient performance through caching, and reliable deployment practices. The design keeps a balance between simplicity and capability, making it suitable for production use. Hybrid search improves accuracy but slightly increases latency. Separate indexes improve security but increase cost. The system prefers shared infrastructure unless strict isolation is required.